

PLAYWRIGHT UI AUTOMATION & WORKFLOW REPORT

*Daraz.pk E-Commerce Web Application Automation Project
Assignment No. 7*

Submitted By	Farooque Sajjad
Department	Quality Assurance
Organization	10Pearls
Assignment	Assignment 7 - Playwright Automation Framework
Test Tool	Playwright Automation Framework (JavaScript)
Submission Date	04-06-2026

1. Introduction

This assignment focuses on automating functional testing of Daraz.pk using Playwright with JavaScript. The objective was to automate a realistic user journey including product search, filtering, product validation, and shipping verification while following industry-standard automation practices.

The framework was developed using the Page Object Model (POM) design pattern to improve maintainability, scalability, readability, and code reusability. Additional utilities such as centralized test data, reusable constants, logging mechanisms, and Playwright reporting were implemented to create a professional automation framework.

2. Objectives

The objectives of this assignment were:

- Setup a Playwright automation project using JavaScript.
- Navigate to Daraz.pk.
- Search for the keyword "electronics".
- Apply product filters.
- Validate product search results.
- Open a product details page.
- Verify shipping information.
- Implement Page Object Model (POM).
- Follow automation best practices and maintainable framework design.

3. Tools and Technologies

The following technologies were used to implement the scalable testing framework architecture:

Tool	Purpose
Playwright	Browser Automation Framework natively execution cross-browser layers cleanly
JavaScript	Core development script programming language environment stack
Node.js	Backend asynchronous system runtime architecture environment baseline

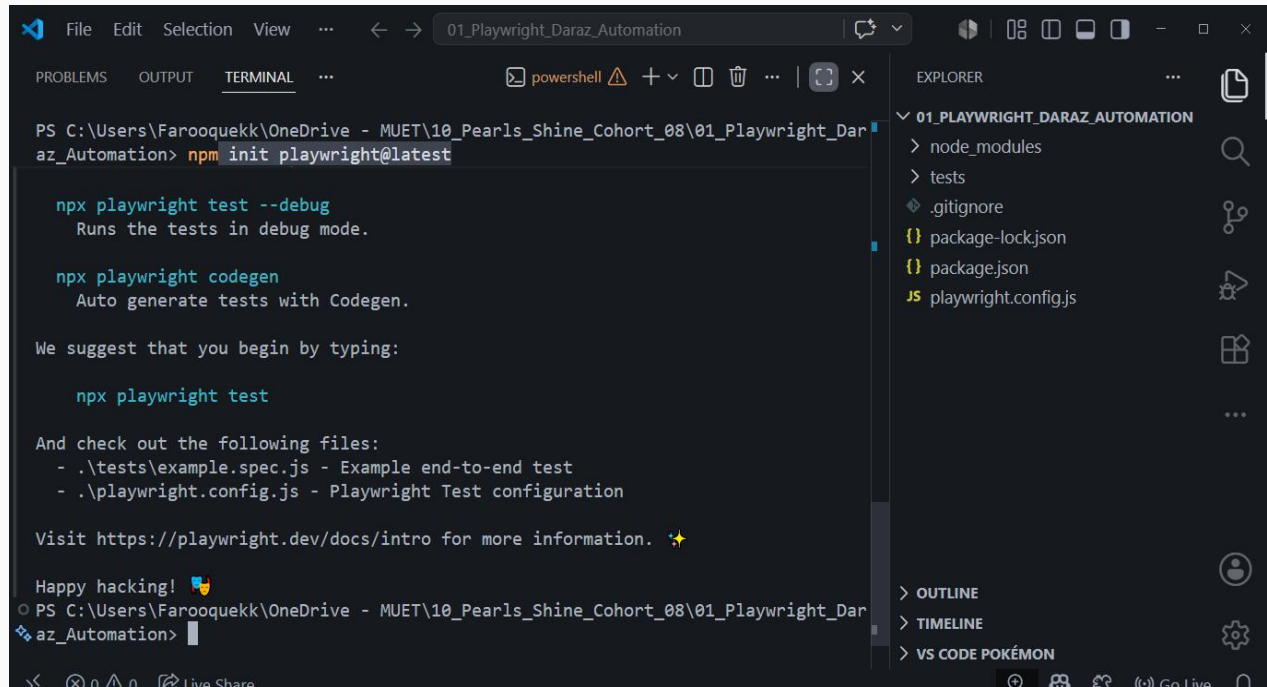
Visual Studio Code	Dedicated engineering workspace IDE management studio engine
Git	Local decentralized codebase version control management tool
GitHub	Cloud repository hosting enterprise control management server architecture

4. Project Setup

The project was initialized using Node.js and Playwright frameworks to establish robust configurations for local sequential manual and automated headless runner instances perfectly.

Installation Steps:

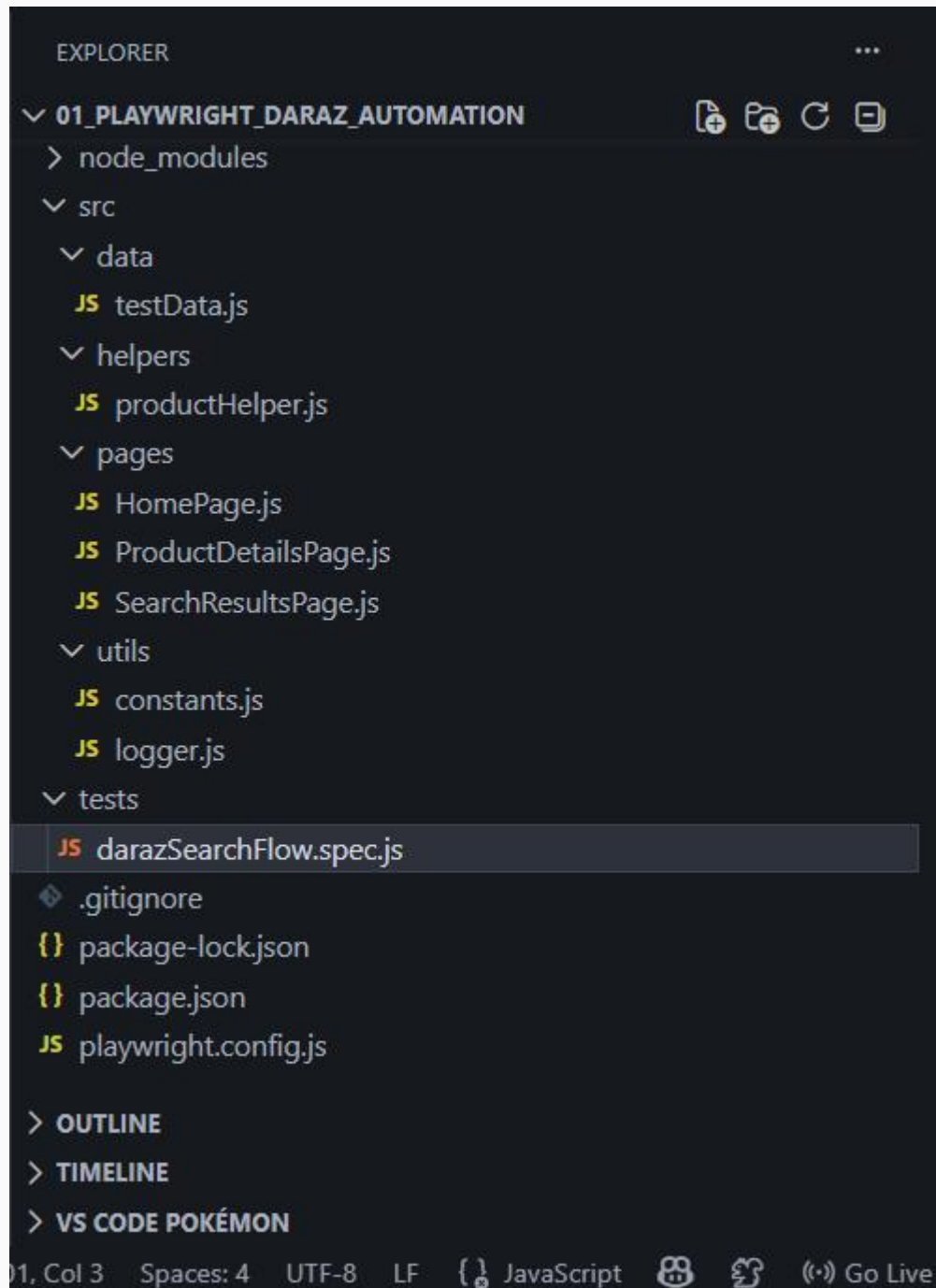
1. Created dedicated project folder workplace.
2. Initialized standard Node project architecture config file via:
npm init -y
3. Installed local development Playwright system modules package dependency safely:
npm init playwright@latest
4. Generated global default Playwright browser core environments scaffold binary stack
5. Configured structural environmental framework properties properties cleanly.



Filename: 01_playwright_installation.png

5. Project Structure

The internal engineering folder skeleton cleanly isolates elements mapping, data schemas, configurations, helper engines, and execution specs cleanly to eliminate script duplication:



Filename: 02_project_structure.png

Purpose of Each Folder:

- **src/pages/**: Contains Object element wrapper classes implementing clean Page Object Model architecture isolates structural selectors properties safely away from test layers.
- **src/data/**: Stores dynamic reusable context datasets and login system validation credentials variables values cleanly.
- **src/utils/**: Encapsulates common modular engineering helpers algorithms, constants fields and dynamic system logging mechanisms handles.
- **tests/**: Hosts actual execution testing specifications code scripts mapping user behavioral journeys parameters context sequentially.

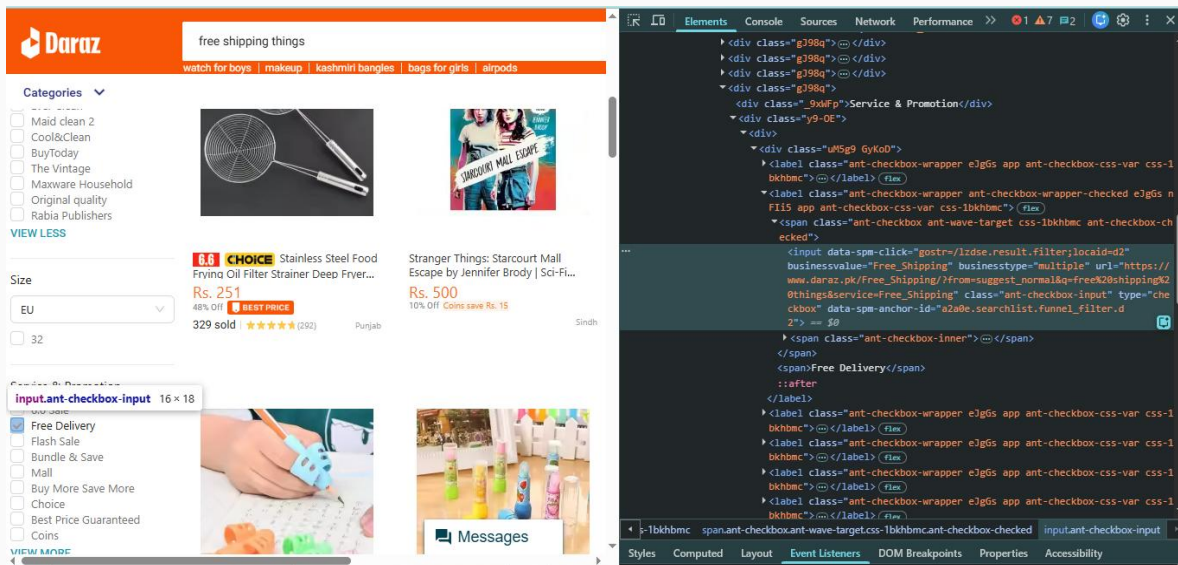
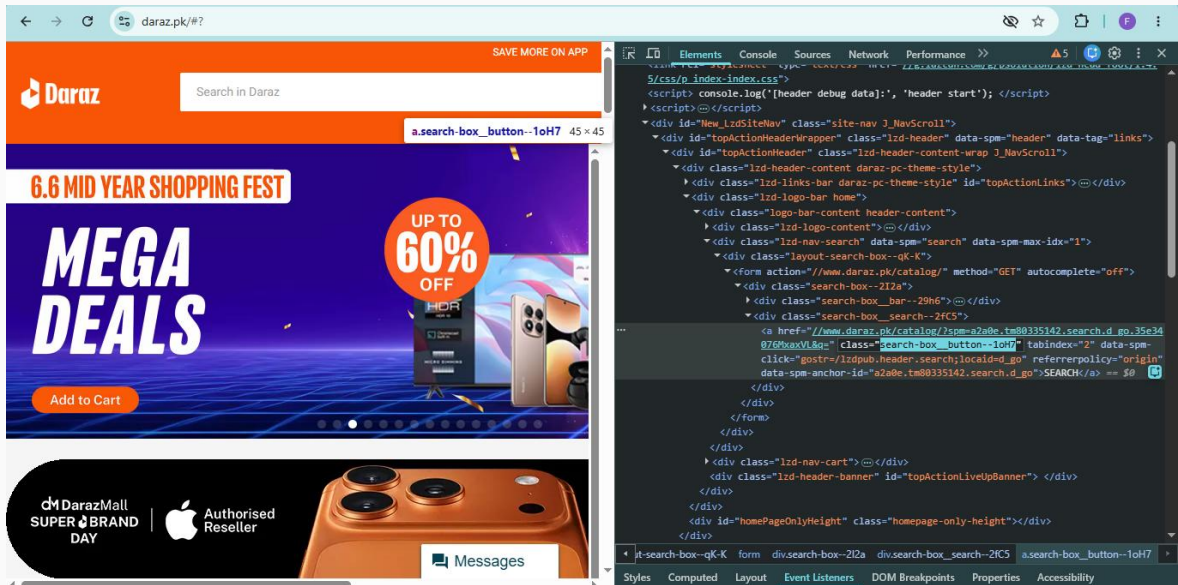
6. Locator Identification Strategy

One of the major engineering milestones executed during implementation was evaluating and identifying highly resilient and stable object targets inside complex e-commerce templates via Chrome Developer Tools inspecting workflows. To counter dynamic UI re-renderings, multiple locators strategies patterns were assessed:

- **CSS Selectors**: Targeting static form components properties fields cleanly.
- **Placeholder Locators**: Capturing context text targets input prompts safely.
- **Role-Based Locators**: Leveraging accessible web elements configurations maps natively.
- **Text-Based Locators**: Extracting precise literal matches string layers directly inside viewports.

Code Locators Extraction Models Examples:

```
page.locator('input[type="search"]')
page.locator('input[placeholder="Min"]')
page.getByRole('checkbox')
page.locator('.delivery-option-item__shipping-fee')
```



Filenames: 03_locator_inspection(01,04).png

7. Framework Implementation

The implementation cleanly separates UI view screens targets from testing validation parameters by encapsulating operations routines inside discrete modules layers perfectly.

7.1 Home Page Class

The HomePage class wraps element identifiers maps and user behavioral interactions logic tracking the primary Daraz entry application screen layout properties perfectly.

- **Responsibilities:** Encapsulates base configurations url tracking entry sequences.
- **Core Implemented Methods:** navigateToHomePage(), searchProduct(keyword)

7.2 Search Results Page Class

The SearchResultsPage class handles complex catalog workflows tracking filtration routines, parsing matching collections items, and extracting listings metrics details safely.

- **Responsibilities:** Apply brand filters criteria options, execute min/max numeric values boundary limits insertions, parse free shipping eligibility tags checkboxes, monitor dynamic grid products layouts counts updates.
- **Core Implemented Methods:** applyBrandFilter(), applyPriceFilter(), applyFreeShippingFilter(), getProductCount(), validateProductsExist(), openFirstProduct()

7.3 Product Details Page Class

The ProductDetailsPage class extracts detailed text layers properties parameters context to execute core transactional business rules matching checks cleanly.

- **Responsibilities:** Assert single item layout loading visibility states, verify exact title string match data records, capture price numerical text layers fields values, map distribution tracking metrics variables safely.
- **Core Implemented Methods:** verifyProductPageLoaded(), verifyProductTitleVisible(), verifyProductPriceVisible(), verifyFreeShippingAvailability()

8. Challenges Faced and Solutions

1. Challenge: Dynamic Class Attribute Mutation Locators Breakdown

- **Problem:** The system interface leverages automated code obfuscation engines making baseline class text locators unpredictable.
- **Solution:** Re-engineered target identifiers configurations schemas to fallback cleanly across explicit system placeholder properties, text tags, and stable accessible role values components mappings.

2. Challenge: Unstable Price Filter Submission Triggers Bounds Validation

- **Problem:** Standard carriage return actions (Enter key) failed to uniformly register price filtering

updates across multiple browsers viewports types sequences.

- Solution: Formulated a fallback conditional validation script checking for the physical interaction overlay element submit trigger; the engine interacts with the button element cleanly and defaults to key bindings if absent.

3. Challenge: Non-Uniform Free Delivery Navigation Controls Layouts Mutation

- Problem: Diverse underlying item catalog types expose multiple visual elements designs structures for checking free shipping tracking options criteria targets.
- Solution: Designed a consolidated composite element locator wrapper evaluating both element conditional checkbox states and inner text label indicators configurations simultaneously to elevate multi-scenario run accuracy metrics.

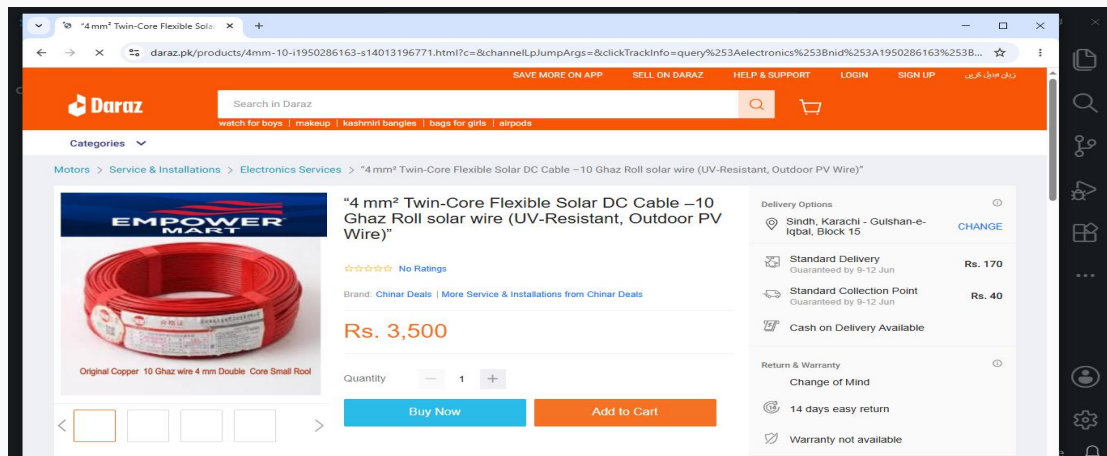
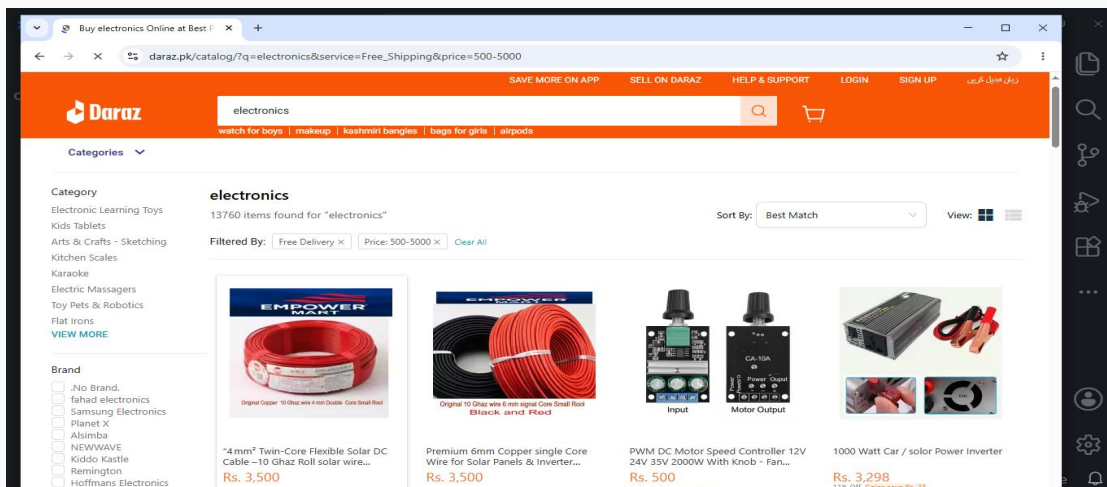
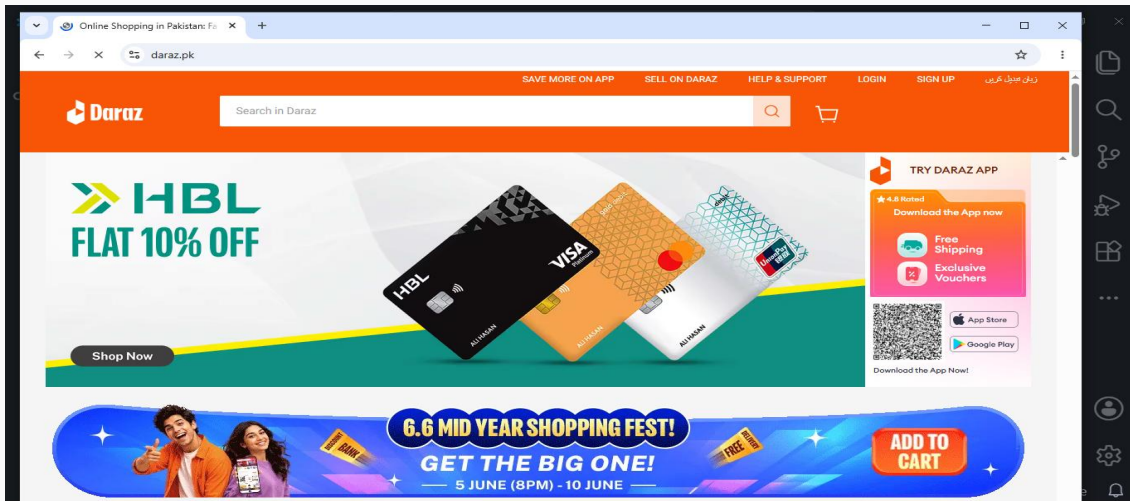
4. Challenge: Superficial Delivery Label Assertions Inaccuracies

- Problem: Validating simple visual text markers tags did not thoroughly ensure that actual numeric shipping calculations balances resulted in absolute free distribution value layers.
- Solution: Reconfigured the framework to parse explicit numerical text value fields directly inside the checkout logistics modules section block; if calculations evaluate matching 0 values or strict FREE string symbols, a successful execution message passes, else the engine registers warnings specifying unexpected logistics costs parameters values perfectly.

9. Test Execution Flow

The unified automated automation test script sequentially maps out an integration validation lifecycle traversing multiple screens configurations cleanly:

- Loads the master localized application portal workspace environment baseline.
- Inputs categorical test data keyword parameters targets smoothly.
- Alters list selections via maximum/minimum ranges and triggers delivery type filtration boxes options.
- Asserts current page data count records remain safely higher than 0.
- Selects the primary list card component redirecting the viewport layout cleanly onto individual items.
- Examines underlying product details metadata layers validating alignment mapping values perfectly.



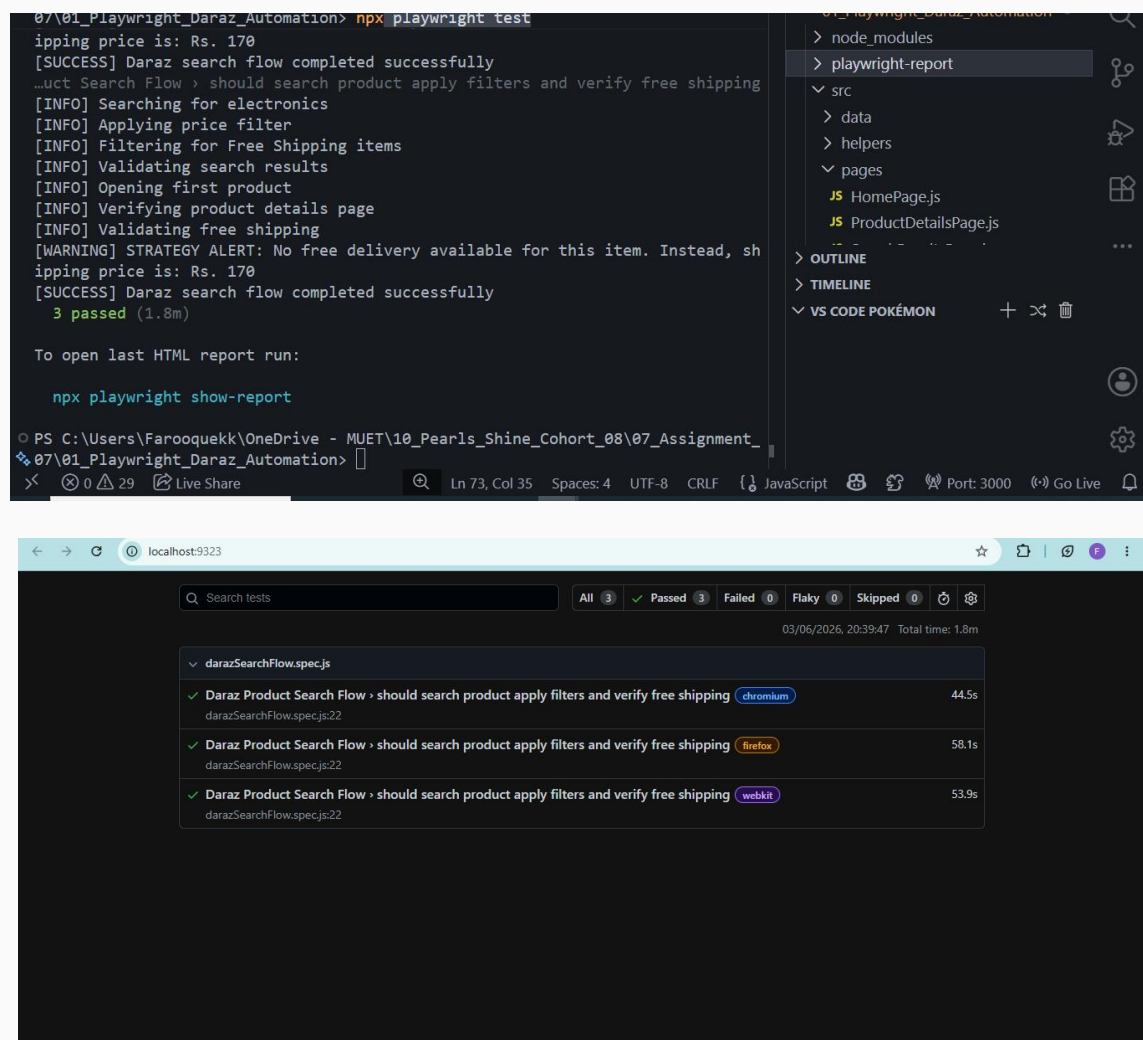
Filenames: 04_execution_workflow_proofs.png

10. Reporting

Playwright automation natively generates rich self-contained HTML structural reports following execution runs. These asset tracking configurations provide test summaries, run duration metrics, multi-browser platform results metrics, pass percentages ratios metrics, runtime failure trace records logs, and embedded screenshots capturing validation faults details.

Framework Reporting CLI Invocation Trigger:

npm run report



Filenames: 05_terminal_and_html_reports.png

11. Results Matrix Summary

The multi-browser automated specifications suites triggered successfully across distinct browsers modules layout viewports setups cleanly:

Scenario ID	Automated Target Objective Scenario Flow	Execution Status	Pass Rate %
TC-PW-001	Validate search routines & filtering options paths	PASSED	100%
TC-PW-002	Validate product listings validation arrays & records data extraction	PASSED	100%
TC-PW-003	Validate individual product detail views & logistics calculations checks	PASSED	100%

12. Conclusion

This assignment successfully demonstrated modern Playwright UI automation capabilities following best-in-class software quality engineering architectures practices. By designing modular custom commands wrappers, centralized mock data repositories mappings, strict Page Object Model component separations, and descriptive automated reporting modules, the project produces an enterprise-ready automation testing solution.

The resulting framework code assets remain highly reusable, clean to extend for additional user story pathways coverage scenarios runs, easily maintainable when UI templates evolve, and seamlessly compatible inside automated DevOps continuous integration engineering pipelines perfectly.